

第6章 优先队列(堆)

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**优先队列** ADT(insert/deleteMin/findMin)与第3章普通队列的区别:出队按优先级而非到达顺序
- 掌握二叉堆的**结构性性质**(完全二叉树)与**堆序性质**(父 $\leq$ 子)
- 掌握 1 基数组表示(根=1、左子 $2i$ 、右子 $2i+1$ 、父 $i/2$),能默写**上滤**与**下滤**
- 能解释两种建堆方式的复杂度差异
- 能说出 d-堆、左式堆、斜堆、二项队列的动机与差别
- 会用 `std::priority_queue` 构造大顶堆、最小堆与自定义比较器
- 能用"大小为 k 的小顶堆"独立解决 top-k,并用标准库验证手写堆

💡 从 C 到 C++:本章主题如何迁移

优先队列的直觉一句话: "不用排队,按重要性插队"。第3章的普通队列是"先来先服务",而急诊分诊、任务调度、短作业打印都要求"最紧急者先处理"。它只有三个操作:insert、findMin、deleteMin。

在 C 里,朴素做法两头为难:无序数组 insert $O(1)$ 但 deleteMin 要扫描 $O(n)$;有序数组反过来。二叉堆的巧妙之处在于 insert 与 deleteMin 同时做到 $O(\log n)$,且不需要指针——完全二叉树直接摊平进数组,父子关系用下标计算,正是 C 程序员最熟悉的数组思维。

C++ 迁移分三层:底层用第3章的 `vector` 当堆数组;中间层手写二叉堆类(\$6.2-\$6.4);上层用标准库 `priority_queue` (\$6.6)。注意 C++ 教材用 1 基(下标 0 闲置),Python 的 `heapq` 用 0 基(\$6.2.3)。

📦 前置知识自查

数学前置:完全二叉树高度 $\approx \log_2 n$;求和记号 Σ 与几何级数(建堆 $O(n)$ 推导用);摊还分析直觉(第11章)。

C 语言前置:数组与下标运算; `struct` 与函数指针;函数指针形式的比较函数;整数除法(1 基父结点 $i/2$)。

依赖前面章节:第2章大 O ;第3章 `vector` (堆的底层数组)与栈;第4章树术语(父、子、叶、深度、高度)。

🚀 本章学习路线

- 第一阶段(1 小时):**读 \$6.1-\$6.2,理解 ADT 与两条性质,背熟 1 基公式
- 第二阶段(1.5 小时):**读 \$6.3-\$6.4,手写上滤/下滤/建堆代码,手工走一遍 insert 与 deleteMin
- 第三阶段(1 小时):**读 \$6.5-\$6.6,俯瞰堆家族,上机实验三种构造
- 第四阶段(实验,1 小时):**完成章末实验:Top-K 高频词
- 验收标准:**能默写上滤与下滤,说出"二叉堆 insert/deleteMin $O(\log n)$ 、建堆 $O(n)$ ",解释 `priority_queue` 为何默认大顶堆

本章知识导图

- §6.1 优先队列模型与动机(任务排程/急诊分诊、ADT 三操作、队列对比)
- §6.2 二叉堆:结构性质与数组表示(完全二叉树、堆序性质、1 基 vs 0 基换算)
- §6.3 插入与删除最小:上滤与下滤(完整代码、"末元素搬根再下滤"套路)
- §6.4 建堆与其余操作(逐个插入 $O(n \log n)$ vs 下滤建堆 $O(n)$ 、decreaseKey/delete/merge)
- §6.5 其他堆:d-堆、左式堆、斜堆与二项队列(动机与一句话差别)
- §6.6 标准库:priority_queue(默认大顶堆、最小堆写法、自定义比较器)
- §6.7 手写堆与标准库并排:top-k 实战(小顶堆淘汰技巧、堆排序预告)

§6.1 优先队列模型与动机

6.1.1 三个场景:谁最紧急谁先走

先看三个场景。① **操作系统任务调度**:每次选择优先级最高的进程;② **打印机队列**:短作业优先;③ **急诊分诊**:按病情严重程度处理,最危重者最先抢救。

三个场景的共同点一致:插入一个元素、查看当前最紧急的元素、取出该元素。大小关系代表优先级还是病情,由具体问题映射,数据结构不管。

6.1.2 优先队列 ADT:三个操作

优先队列(priority queue) 是抽象数据类型: `insert(x)` 插入; `findMin()` 返回最小元但不删除; `deleteMin()` 删除并返回最小元。约定数值越小优先级越高(最小堆),取最大则把比较方向反过来(§6.6)。注意:**堆(heap)** 是优先队列最主流的实现,不是 ADT 本身;与存放动态内存的"堆内存"同名,毫无关系。

记号说明(本章约定,与 Python 版一致)

复杂度记号沿用第2章。下标公式中,"父 $i/2$ "与" $(i-1)/2$ "都指**整数除法**(代码写 $i / 2$ 、 $(i - 1) / 2$)。"最小元""最高优先级"同义。

6.1.3 与普通队列对比:出队顺序由谁决定

第3章的普通队列是 FIFO:出队顺序 = 到达顺序;优先队列相反:出队顺序 = 优先级顺序。朴素实现却两头为难:

朴素实现	insert	deleteMin	问题
无序数组/链表	$O(1)$	$O(n)$ (扫描)	删除太慢
有序数组/链表	$O(n)$ (移位)	$O(1)$	插入太慢
二叉堆(本章主角)	$O(\log n)$	$O(\log n)$	两者均衡

C 的写法	C++ 的写法
<code>struct PQ { int *a; int n; };</code> 数据与操作分离	<code>class BinaryHeap { vector<T> a; ... };</code> 数据与操作封装
<code>pq_deleteMin(&pq, &out);</code> 每个操作传指针	<code>pq.deleteMin();</code> 成员函数天然绑定
找最小要自己写 <code>for</code> 循环扫描	根就是最小元, <code>findMin()</code> 一行
数组长度、容量、释放全靠手工	<code>vector</code> 自动管理

§6.2 二叉堆:结构性质与数组表示

6.2.1 结构性质:完全二叉树

二叉堆首先是一棵**完全二叉树**:除最后一层外每层都满,最后一层从左到右连续填充、不留空隙。这条性质的威力在高度:结点数 n 满足 $2^h \leq n < 2^{h+1}$, 所以 $h \approx \log_2 n$ ——任何根到叶路径都只有对数长度,这正是上滤/下滤高效的根源。

完全二叉树还不需要指针:结点按层序编号,父子关系用下标直接算出(§6.2.3)。对比第4章的指针树,连续数组存储节省指针空间、缓存友好,代价是形状被锁定,只能在末尾增删。

6.2.2 堆序性质:最小堆

堆序性质:对每个非叶结点,父结点的值 $\leq$ 子结点的值 (最小堆;换成 $\geq$ 即最大堆)。两条性质合起来才是二叉堆:结构性质保证对数高度,堆序性质保证根是全局最小元, `findMin()` 是 $O(1)$ 。

立刻澄清:堆序只约束"父子",兄弟之间没有大小关系。堆因此不是有序数组,不能二分查找,按值查找只能全扫 $O(n)$ 。堆只承诺最小元在根,是"弱有序"结构,与第4章 BST 的强有序对照。

6.2.3 数组表示:1 基与 0 基换算

把完全二叉树按层序放进数组,父子关系退化为下标运算。这里出现全系列最重要的"习惯分歧":C++ 教材(Weiss 原书)用 1 基——根在下标 1,下标 0 闲置;Python 的 `heapq` 用 0 基——根在下标 0。两套公式:

含义	1 基(C++ 教材习惯)	0 基(Python <code>heapq</code>)
根	下标 1	下标 0
结点 i 的左子	$2i$	$2i+1$
结点 i 的右子	$2i+1$	$2i+2$
结点 i 的父	$i/2$ (整数除法)	$(i-1)/2$ (整数除法)
最后一个非叶结点	$n/2$ (整数除法)	$n/2 - 1$
下标 0 是否闲置	是, <code>a[0]</code> 闲置	否, 下标 0 就是根

验证:下标 5 的结点,1 基下左子 10、右子 11、父 2;0 基下左子 11、右子 12、父 2。父相同只是巧合——下标 4:1 基父 2,0 基父 1。

⚠ 误区 1:把 0 基公式抄进 1 基代码(或反之)

两种下标体系并存是本章第一大坑。1 基堆里左子必须写 $2*i$ 、父必须写 $i/2$ ——抄成 0 基的 $2*i+1$ 、 $(i-1)/2$ 会取到错位置;把闲置的 $a[0]$ 当根读写,堆序立即被破坏且极难排查。解决之道:写代码前先声明"用几基"并写进注释,写完用"验证堆序"函数检查 (§6.3.3)。

示例 6-1 二叉堆类骨架(1 基,vector 作底层)

```
// 1 基堆:根在下标 1,左子 2i、右子 2i+1、父 i/2;下标 0 闲置
template <typename T>
class BinaryHeap {
    vector<T> a;                // 底层数组,a[0] 闲置
public:
    BinaryHeap() { a.push_back(T{}); } // 占住 a[0],保证从下标 1 开始用
    size_t size() const { return a.size() - 1; }
    bool empty() const { return size() == 0; }
    T findMin() const { return a[1]; } // 根即最小元 0(1)
};
```

§6.3 插入与删除最小:上滤与下滤

6.3.1 插入:上滤(percolateUp)

插入策略是"先保结构,再修堆序":① 新元素追加到末尾(完全二叉树只在末尾留空位,结构自动保持);② 沿根路径向上冒泡——与父比较,小于父则交换,直到父 $\leq$ 自己或到根。这叫**上滤(percolate up)**,路径最长即树高,insert 为 $O(\log n)$;上滤只看一个父,正是堆"弱有序"的便利。

示例 6-2 insert:上滤

```
void insert(const T& x) {
    a.push_back(x);        // 追加到末尾:保持完全二叉树
    size_t i = a.size() - 1; // 新元素下标
    while (i > 1) {         // 向上冒泡(上滤)
        size_t p = i / 2;   // 父结点(1 基)
        if (a[p] <= a[i]) break; // 父 <= 子,堆序成立,停
        swap(a[p], a[i]);   // 否则与父交换,继续往上
        i = p;
    }
}
```

6.3.2 删除最小:下滤(percolateDown)

删除根会留下空洞,朴素前移会破坏完全二叉树或付出 $O(n)$ 搬移。经典套路:① 记下根(它就是答案);② 把**最后一个元素搬到根**(末尾是唯一可合法删除的空位,结构自动保持);③ 沿"叶方向"向下冒泡——与孩子中**较小者**比较,大于它则交换,直到 $\leq$ 两个孩子或到叶。这叫**下滤(percolate down)**,同样 $O(\log n)$ 。

两个细节:必须与"较小的"孩子交换(与较大的换会违反堆序);先检查右子是否存在(下标 $2i+1$ 是否在界内),只有左子也合法。

示例 6-3 deleteMin:末元素搬根再下滤

```

// 下滤:把下标 i 的元素向下沉(deleteMin 与建堆共用)
void percolateDown(size_t i) {
    size_t n = a.size();
    while (true) {
        size_t smallest = i;
        for (size_t c : {2 * i, 2 * i + 1}) // 左子、右子(1 基)
            if (c < n && a[c] < a[smallest])
                smallest = c; // 选"较小"的孩子
        if (smallest == i) break; // 两个孩子都不小于自己
        swap(a[i], a[smallest]);
        i = smallest;
    }
}

// 删除最小元:最后一个元素搬到根,再下滤
T deleteMin() {
    T min = a[1]; // 记下根(答案)
    a[1] = a.back(); // 末元素搬到根:保持完全二叉树
    a.pop_back();
    if (a.size() > 1) percolateDown(1); // 只剩 1 个元素则免
    return min;
}

```

例题 6-1 手工走一遍 insert 与 deleteMin

设 1 基堆初始为 $[_, 13, 21, 16, 24, 31, 19, 68, 65, 26, 32]$ ($a[0]$ 闲置)。**insert(14)**:追加到下标 11,沿 $11 \rightarrow 5 \rightarrow 2 \rightarrow 1$ 上滤: $31 > 14$ 换、 $21 > 14$ 换、 $13 \leq 14$ 停,得 $[_, 13, 14, 16, 24, 21, 19, 68, 65, 26, 32, 31]$ 。**deleteMin()**:取走 13,末元素 31 搬根,依次换过 14、21、26,得 $[_, 14, 21, 16, 24, 26, 19, 68, 65, 31, 32]$,最小元 14。

6.3.3 完整接口与调试技巧

完整堆还应有 `top()/findMin()`、`empty()`、`size()` 等接口;若需"按插入顺序稳定",可在元素里加序号比较(见 §6.7 的(频次, 词)元组)。

调试技巧:写一个"验证堆序"的断言函数

堆的 bug 极具隐蔽性:上滤少换一层、下滤选错孩子,结果仍"接近正确"。随手加 `checkHeap()`:遍历所有非叶结点 $i(1 \leq i \leq n/2)$,断言 $a[i] \leq a[2*i]$ (若存在)且 $a[i] \leq a[2*i+1]$ (若存在),每次 insert/deleteMin 后调用,把可疑行为扼杀在摇篮里。

§6.4 建堆与其余操作

6.4.1 两种建堆方式

建堆有两种做法。① **自顶向下**:空堆逐个 insert,每次 $O(\log n)$,共 $O(n \log n)$ 。② **自底向上(下滤建堆)**: n 个元素按任意顺序放进数组(天然构成完全二叉树,只是堆序可能被违反),从最后非叶结点($n/2$)往前逐个 `percolateDown` 到根,总代价 $O(n)$ 。"一次性给全所有元素"的场景都用下滤建堆。

示例 6-4 下滤建堆: $O(n)$

```
// 下滤建堆:从最后一个非叶结点(n/2)开始,逐个下滤到根
void buildHeap(vector<T> items) {
    a.clear();
    a.push_back(T{});          // 占位 a[0]
    for (const T& x : items) a.push_back(x);
    for (size_t i = a.size() / 2; i >= 1; --i)
        percolateDown(i);      // 复用示例 6-3 的下滤
}
```

例题 6-2 手工走一遍下滤建堆

输入 [150, 80, 40, 30, 10, 70, 110, 100, 20, 90, 60, 50, 120, 140, 130] ($n = 15$, 最后非叶 = 下标 7)。
 从 7 往前: 7(110) ≤ 130 停; 6(70) 与 50 换; 5(10) ≤ 90、60 停; 4(30) 与 20 换; 3(40) ≤ 50、110 停; 2(80) 与 10 换、再与 60 换; 1(150) 依次与 10、20、30 换。最终 [_, 10, 20, 40, 30, 60, 50, 110, 100, 80, 90, 70, 150, 120, 140, 130]。

6.4.2 为什么下滤建堆是 $O(n)$:一句话推导

关键观察:高度为 h 的结点约有 $n/2^{h+1}$ 个,每个下滤至多走 h 步,总代价 $\sum_h h \cdot n/2^{h+1}$ 是"几何级数收尾"的求和,结果 $O(n)$ 。直觉:矮的结点数量多但只需下滤几步,高的结点上滤远但数量极少;逐个插入的代价全集中在"深叶子上滤到根"的长路径。严格证明在第11章。

6.4.3 decreaseKey、delete 与 merge

- **decreaseKey(p, delta)**:键变小只能向上走——直接上滤, $O(\log n)$ 。前提是知道 p 的位置(堆不提供查找);经典用途:Dijkstra 松弛(第9章)。
- **delete(任意位置 p)**:把 $a[p]$ 换成最小值(根的副本),上滤到根,再 deleteMin——两步都 $O(\log n)$ 。
- **merge(合并两个堆)**:二叉堆的短板——必须把两个数组拷到一起再建堆, $O(n)$ 。频繁合并堆的应用,二叉堆不够用,这正是 §6.5 堆家族的动机。

示例 6-5 decreaseKey 与 delete(任意位置)

```
// 键变小只会"向上"走:减键后上滤  $O(\log n)$ 
void decreaseKey(size_t pos, const T& amount) {
    a[pos] -= amount;
    while (pos > 1 && a[pos / 2] > a[pos]) {
        swap(a[pos / 2], a[pos]);
        pos /= 2;
    }
}

// 删除任意位置:盖成最小值上滤到根,再 deleteMin  $O(\log n)$ 
void removeAt(size_t pos) {
    a[pos] = a[1];          // 根是最小值,盖上去不破坏堆序
    while (pos > 1) {        // 一路换到根
        swap(a[pos / 2], a[pos]);
        pos /= 2;
    }
    deleteMin();            // 现在根就是原来的 a[pos]
}
```


§6.5 其他堆:d-堆、左式堆、斜堆与二项队列

6.5.1 为什么不止一种堆

二叉堆已做到 insert/deleteMin 均 $O(\log n)$ 、 findMin $O(1)$,为何还要别的堆?因为**操作组合不同**:插入远多于删除的系统希望 insert 更快;频繁合并堆的系统受不了 merge $O(n)$ 。没有一种堆能同时最优,每个成员都在特定操作上让步、换取另一操作的改进。

6.5.2 d-堆:多叉,降低高度

d-堆 是二叉堆的直接推广:每个结点最多 d 个孩子(二叉堆即 $d = 2$)。高度从 $\log_2 n$ 降到 $\log_d n$, insert (只与一个父比较)变快为 $O(\log_d n)$;但 deleteMin 要在 d 个孩子里找最小,变慢为 $O(d \log_d n)$ 。典型场景: insert 远多于 deleteMin 的离散事件模拟,常用 $d = 4$ 或 5 。

6.5.3 左式堆:为 merge 而生

二叉堆 merge $O(n)$ 的根源是"完全二叉树"锁死了形状。**左式堆(leftist heap)** 主动放弃完全性,改用新约束:对每个结点,左子的零路径长(NPL)不小于右子的——即"左偏"。**零路径长(null path length)** 是到最近叶子的最短路径长度(空结点为 -1)。它保证最右路径很短, merge 只需递归沿最右路径合并, $O(\log n)$; insert/deleteMin 都可用 merge 实现。

6.5.4 斜堆:左式堆的摊还简化版

斜堆(skew heap) 是左式堆的极简主义版本:不维护 NPL, merge 后**无条件交换左右子树**。实现更短,但单次可能退化 $O(n)$,整体摊还 $O(\log n)$ (第11章证明)。它与左式堆的关系,正如伸展树之于 AVL 树。

6.5.5 二项队列:森林

二项队列(binomial queue) 的思路完全不同:不是一棵树,而是一组"二项树"组成的**森林**。第 k 棵二项树大小恰为 2^k ,每种大小最多一棵—— n 个元素按二进制位决定有哪些树(如 $n = 13 = 1101$,有 8 、 4 、 1 三棵)。插入像二进制加法"进位"合并同大小树,摊还 $O(1)$; merge 与 deleteMin 均 $O(\log n)$ 。

堆类型	动机	insert	deleteMin	merge
二叉堆	最常用,数组存储、缓存友好	$O(\log n)$	$O(\log n)$	$O(n)$
d-堆	插入多于删除,降高度	$O(\log_d n)$	$O(d \log_d n)$	$O(n)$
左式堆	高效 merge,左偏约束	$O(\log n)$	$O(\log n)$	$O(\log n)$
斜堆	左式堆的摊还简化版	摊还 $O(\log n)$	摊还 $O(\log n)$	摊还 $O(\log n)$
二项队列	森林结构,插入摊还 $O(1)$	摊还 $O(1)$	$O(\log n)$	$O(\log n)$

§6.6 标准库:priority_queue

6.6.1 三种构造:大顶堆、最小堆与自定义比较器

`std::priority_queue` 是优先队列的工程实现,底层默认用 `vector` 存堆。最大的坑:默认是"大顶堆"——`top()` 返回最大元素(底层比较器 `less<T>`)。要最小堆必须写全三个模板参数;自定义比较器语义与 `std::sort` 相反:返回 `true` 表示第一个参数优先级更低。

示例 6-6 priority_queue 的三种构造

```
#include <queue>
#include <functional>
using namespace std;

priority_queue<int> mx;           // ① 默认大顶堆(底层 less):top() 是最大元
mx.push(3); mx.push(1); mx.push(9);
// mx.top() == 9, pop 顺序:9 3 1

// ② 最小堆:三模板参数
priority_queue<int, vector<int>, greater<int>> mn;
mn.push(3); mn.push(1); mn.push(9);
// mn.top() == 1, pop 顺序:1 3 9

// ③ 自定义比较器:按"频次"给 (词, 频次) 建最小堆
struct Cmp {
    bool operator()(const pair<string, int>& x,
                    const pair<string, int>& y) const {
        return x.second > y.second; // 返回 true 表示 x 优先级更低
    }
};
priority_queue<pair<string, int>, vector<pair<string, int>>, Cmp> pq;
```

⚠ 误区 2:忘记 priority_queue 默认是大顶堆

习惯"优先队列 = 最小先出"的同学,直接用 `priority_queue<int>` 拿到的却是最大元——标准库最高频 bug。两个检查:① 想清楚 `top()` 出来的是谁,最小堆务必写全三个模板参数,嫌啰嗦可用 `using` 起别名;② `pop()` / `top()` 前先判 `empty()`,空堆 `top` 未定义。

6.6.2 接口限制与选型

`priority_queue` 接口只有五个:push、pop、top、empty、size。限制:`pop()` 不返回被删元素;无迭代器;无 `decreaseKey`;"批量建堆"只能在构造时拷入整个容器(内部自动下滤)。它本质是**适配器**:包住一个 `vector` 只露出堆的三操作,与 `std::stack` / `std::queue` 同一家族。

C 的手写二叉堆	C++ 的 std::priority_queue
自己写数组、上滤、下滤约 60 行	构造一行声明,五个成员函数即全接口
比较逻辑写死在代码里	比较器是模板参数,less/greater/自定义随意换
内存泄漏风险全在自己肩上	RAII 自动管理,离开作用域自动释放
可以随时读写底层数组(调试友好)	封装后只能走三操作,防止误用

🚀 工程建议:什么时候直接上 priority_queue

工程中"需要动态维护最值"的场景一律优先 `priority_queue`:任务调度、Top-K、贪心(第10章哈夫曼)、Dijkstra(第9章)。手写堆的用武之地:需要 `decreaseKey`(标准库没有)、批量建堆后长期复用、或考试要求展示原理。"一次性排好序"用 `std::sort`;"边插入边取最值"才用堆。

§6.7 手写堆与标准库并排:top-k 实战

6.7.1 问题:求最大的 k 个(top-k)

Top-K 问题:给定 n 个元素,求最大的 k 个(排行榜、热门词)。朴素做法全部排序是 $O(n \log n)$;堆做法 $O(n \log k)$, k 远小于 n 时近乎线性。核心技巧叫**小顶堆淘汰法**:维护"当前最大的 k 个"的小顶堆——堆顶恰是第 k 大,即淘汰"门槛";新元素大于堆顶才替换。

为什么用"小顶堆"而非"大顶堆"? 大顶堆的堆顶是全局最大,淘汰后不知谁替补;小顶堆的堆顶是"最小的入选者",谁比它大谁挤掉它,淘汰逻辑一行搞定。

C 的朴素做法(qsort 全排序)	C++ 的堆做法(priority_queue)
qsort 把 n 个元素全部排序, $O(n \log n)$	大小为 k 的小顶堆, $O(n \log k)$
结果数组占 n 个位置	堆只占 k 个位置,可流式处理
新数据到达必须重新 qsort 一遍	push / top / pop 动态维护,每次 $O(\log k)$
比较函数用函数指针,写在调用处之外	比较器是模板参数, <code>greater<T></code> 一行搞定

⚠ 误区 3:top-k 时"无脑入堆",忘了先与堆顶比较

堆满后仍直接 push,堆会涨过 k ;或"先 push 再 pop 堆顶",把刚进来的大元素又弹走——淘汰条件写反。正确顺序只有一种:先与堆顶比较,大于门槛才 pop 再 push,且 pop 必须在 push 之前。把"比较 → pop → push"的顺序写成注释即可防错。

6.7.2 手写版与标准库版并排

同一任务——给一组(频次, 词)求频次最高的 k 个——分别用手写堆与 priority_queue 实现。两份代码逻辑结构一字不差,正是"手写理解原理、标准库交付工程"的教科书式对照。

示例 6-7 手写堆版 top-k(复用示例 6-1~6-3 的 BinaryHeap)

```
// items: (频次, 词);按频次降序输出最大的 k 个
vector<pair<int, string>> topK(const vector<pair<int, string>>& items, int k) {
    BinaryHeap<pair<int, string>> h;           // 手写最小堆
    for (const auto& t : items) {
        if (h.size() < size_t(k))
            h.insert(t);                      // 堆未满:直接入堆
        else if (t > h.findMin()) {           // 超过"第 k 大门槛"才替换
            h.deleteMin();
            h.insert(t);
        }
    }
    vector<pair<int, string>> res;
    while (!h.empty()) { res.push_back(h.findMin()); h.deleteMin(); }
    reverse(res.begin(), res.end());          // 弹出是升序,反转为降序
    return res;
}
```

示例 6-8 标准库版 top-k:逻辑一字不差

```
// 同一任务用 priority_queue:只换构造与函数名
vector<pair<int, string>> topKStd(const vector<pair<int, string>>& items, int k) {
    using P = pair<int, string>;
    priority_queue<P, vector<P>, greater<P>> h;    // 标准库最小堆
    for (const P& t : items) {
        if (int(h.size()) < k)
            h.push(t);
        else if (t > h.top()) {                    // 同上:高于门槛才替换
            h.pop();
            h.push(t);
        }
    }
    vector<P> res;
    while (!h.empty()) { res.push_back(h.top()); h.pop(); }
    reverse(res.begin(), res.end());
    return res;
}
```

6.7.3 复杂度分析与堆排序预告

每个元素至多一次比较 + 一次替换($O(\log k)$),总代价 $O(n \log k)$; $k = n$ 时退化为 $O(n \log n)$ 。优势在**内存**:堆只占 k 个位置,几十 GB 的日志放不进内存,却能流式处理。

预告第7章:堆排序直接复用本章的二叉堆——建堆 $O(n)$,连续 deleteMin n 次,弹出元素天然有序,总代价 $O(n \log n)$,与插入、希尔、归并、快排逐一对比。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
insert	二叉堆	$O(\log n)$	$O(\log n)$
deleteMin / deleteMax	二叉堆	$O(\log n)$	$O(\log n)$
findMin	二叉堆	$O(1)$	$O(1)$
buildHeap(下滤建堆)	二叉堆	$O(n)$	$O(n)$
buildHeap(逐个插入)	二叉堆	$O(n \log n)$	$O(n \log n)$
decreaseKey	二叉堆	$O(\log n)$	$O(\log n)$
delete(任意元素)	二叉堆	$O(\log n)$	$O(\log n)$
merge	二叉堆	$O(n)$	$O(n)$
按值查找 find	二叉堆	$O(n)$	$O(n)$
insert	d-堆	$O(\log_d n)$	$O(\log_d n)$
deleteMin	d-堆	$O(d \log_d n)$	$O(d \log_d n)$
merge	左式堆	$O(\log n)$	$O(\log n)$
merge	斜堆	摊还 $O(\log n)$	$O(n)$
insert	二项队列	摊还 $O(1)$	$O(\log n)$
deleteMin	二项队列	$O(\log n)$	$O(\log n)$
merge	二项队列	$O(\log n)$	$O(\log n)$
top-k(大小为 k 的小顶堆)	二叉堆 + 一次遍历	$O(n \log k)$	$O(n \log k)$

最小必会清单

- 能说出优先队列 ADT 的三个操作与普通队列的本质区别
- 能说出二叉堆的两条性质,并画出完全二叉树的数组填充
- 能默写 1 基公式(左子 $2i$ 、右子 $2i+1$ 、父 $i/2$ 、最后非叶 $n/2$),并与 0 基公式换算
- 能默写 insert(上滤)与 deleteMin(末元素搬根再下滤)的代码
- 能解释"删除最小为什么是搬最后一个元素",以及"下滤为什么必须选较小的孩子"
- 能独立写出下滤建堆,并解释两种建堆方式的复杂度差异
- 能说出 decreaseKey、delete、merge 的代价与场景
- 能一句话说出 d-堆、左式堆、斜堆、二项队列的动机与差别
- 能写出最小堆 priority_queue 的完整构造,并指出默认大顶堆的坑
- 能独立实现"大小为 k 的小顶堆"解决 top-k,并解释为什么不用大顶堆

自测题

- Q 1. 二叉堆为什么必须是完全二叉树?如果允许任意形状的二叉树,会损失什么?**

✓ 参考答案

完全性保证高度恒为 $\log_2 n$ 与数组连续存储(无指针、下标算父子)。若形状任意,最坏退化成链,高度与操作都变 $O(n)$,还得用指针。

Q 2.1 基堆中下标 i 的结点的父、左子、右子分别是什么?0 基堆呢?试各举一例。

✓ 参考答案

1 基:父 $i/2$ 、左子 $2i$ 、右子 $2i+1$ (如 $i=5$:父 2、左 10、右 11);0 基:父 $(i-1)/2$ 、左子 $2i+1$ 、右子 $2i+2$ (如 $i=5$:父 2、左 11、右 12)。“父为 2”只是巧合,不可混用。

Q 3. 在最小堆中查找某个值的复杂度是多少?为什么不能像 BST 那样二分?

✓ 参考答案

$O(n)$ 。堆序只约束父子关系,兄弟之间没有大小关系,无法一次比较排除一半子树;堆也不是有序数组,连续存储帮不上二分。堆只承诺最小元在根。

Q 4. 把 15 个元素下滤建堆,最后一个下滤的结点是哪个?为什么不是第一个?

✓ 参考答案

最后下滤的是根(下标 1)。建堆从最后非叶结点($n/2$)往前处理,先让下面子树各自成堆,上层下滤时才能保证孩子子树已是堆、只需比较两个孩子。根最后处理,一次下滤把最小值带到根。

Q 5. deleteMin 为什么要把“最后一个元素”搬到根,而不是把后面元素整体前移?

✓ 参考答案

整体前移会留下空隙(破坏结构性)或付出 $O(n)$ 搬移。最后一个元素是唯一“删除后数组仍连续”的位置:先搬到根保持结构,再下滤修复堆序,两步各 $O(\log n)$ 。

Q 6. 求 top-k 为什么用大小为 k 的“小顶堆”?用“大顶堆”行吗?

✓ 参考答案

小顶堆的堆顶是“当前入选者里最小的”(第 k 大门槛),新元素与堆顶比较即可决定淘汰谁。大顶堆的堆顶是全局最大,被淘汰后无法 $O(1)$ 判断谁替补,只能维护更多信息或退化成排序。求“最小的 k 个”改用大小为 k 的大顶堆。

章末实验题:Top-K 高频词(堆实现)

实验 6:Top-K 高频词(手写堆 + 标准库双实现)

实验目标:流式读入英文文本,先用**手写二叉堆**求频次最高的 k 个词,再用 `std::priority_queue` 重做并验证两版结果一致。

实验要求:

- 任务 1:实现二叉堆类(insert/deleteMin/下滤建堆,1 基或 0 基自选,注释中说明所用下标公式)(知识点:\$6.2-\$6.4 上滤/下滤/建堆)
- 任务 2:流式读入文本,切分出单词并维护"词频"字典(知识点:\$6.1 + 第5章散列)
- 任务 3:用"大小为 k 的小顶堆"求频次最高的 k 个词,淘汰技巧:堆满时频次高于堆顶才替换(知识点:\$6.7 top-k)
- 任务 4:用 `priority_queue` (注意最小堆构造)重做任务 3,对比两版结果一致(知识点:\$6.6 标准库)
- 任务 5:处理边界: $k \leq 0$ 、文本为空、 k 大于词数时给出合理输出而非崩溃(知识点:\$6.3)

实验环境:C++17。编译: `g++ -std=c++17 -O2 lab6.cpp -o lab6` ;运行: `./lab6 k < 文本文件` 。

第一步 写 MinHeap 模板类(1 基),用例题 6-1 的序列手工验证 insert 与 deleteMin,再写 top-k 函数

第二步 切分用 `isalpha` 逐字符扫描,全部转小写;词频字典用 `unordered_map<string, int>`

第三步 元素用 `pair<int, string>` (频次在前):先比频次、平局比词序

第四步 准库版只改三处:构造(加 greater)、push/pop 换名、堆顶 `h.top()`;便于对照

第五步 行样例: `./lab6 3 < sample.txt` ,再试 $k = 0$ 与空文件

✓ 参考答案(完整可运行程序,约 120 行,分三段)


```

// lab6.cpp — 第一段:MinHeap(1 基)
#include <algorithm>
#include <cctype>
#include <iostream>
#include <queue>
#include <string>
#include <unordered_map>
#include <vector>
using namespace std;

// 任务 1:手写二叉堆(1 基,下标公式见 §6.2.3)
template <typename T>
class MinHeap {
    vector<T> a; // 元素从下标 1 开始

    void siftDown(size_t i) { // 下滤:与较小孩子交换
        size_t n = a.size();
        while (true) {
            size_t smallest = i;
            for (size_t c : {2 * i, 2 * i + 1}) // 左子、右子(1 基)
                if (c < n && a[c] < a[smallest])
                    smallest = c;
            if (smallest == i) break;
            swap(a[i], a[smallest]);
            i = smallest;
        }
    }

public:
    MinHeap() { a.push_back(T{}); } // 占住 a[0]
    size_t size() const { return a.size() - 1; }
    bool empty() const { return size() == 0; }
    const T& top() const { return a[1]; } // 根即最小元 O(1)

    void push(const T& x) { // 上滤:放末尾后向上冒泡
        a.push_back(x);
        size_t i = a.size() - 1;
        while (i > 1) {
            size_t p = i / 2; // 父结点(1 基)
            if (a[p] <= a[i]) break;
            swap(a[p], a[i]);
            i = p;
        }
    }

    void pop() { // 删除最小:末元素搬根再下滤
        a[1] = a.back();
        a.pop_back();
        if (a.size() > 1) siftDown(1);
    }

    static MinHeap buildFrom(const vector<T>& items) { // 下滤建堆 O(n)
        MinHeap h;
        for (const T& x : items) h.a.push_back(x);
        for (size_t i = h.a.size() / 2; i >= 1; --i) // 最后非叶 = n/2

```

```

        h.siftDown(i);
    return h;
}
};

```

lab6.cpp(第二段:词频统计与手写堆 top-k)

```

// 任务 2:流式切词并统计词频
void countWords(unordered_map<string, int>& freq) {
    string line;
    while (getline(cin, line)) {
        for (size_t i = 0; i < line.size(); i++) {
            if (!isalpha((unsigned char)line[i])) { ++i; continue; }
            size_t j = i;
            while (j < line.size() && isalpha((unsigned char)line[j])) ++j;
            string w = line.substr(i, j - i);
            for (char& c : w) c = char(tolower((unsigned char)c));
            ++freq[w];
            i = j;
        }
    }
}

// 任务 3:手写小顶堆求 top-k
vector<pair<int, string>> topkHeap(const vector<pair<int, string>>& items, int k) {
    MinHeap<pair<int, string>> h;
    for (const auto& t : items) { // t = (频次, 词)
        if (h.size() < size_t(k))
            h.push(t); // 堆未满:直接入堆
        else if (t > h.top()) { // 高于"第 k 大门槛"才替换
            h.pop();
            h.push(t);
        }
    }
    vector<pair<int, string>> res;
    while (!h.empty()) { res.push_back(h.top()); h.pop(); }
    reverse(res.begin(), res.end()); // 弹出是升序,反转为降序
    return res;
}

```

lab6.cpp(第三段:标准库版与主程序)

```

// 任务 4:标准库重做(小顶堆 = 大顶堆 + greater)
vector<pair<int, string>> topkStd(const vector<pair<int, string>>& items, int k) {
    using P = pair<int, string>;
    priority_queue<P, vector<P>, greater<P>> pq;
    for (const P& t : items) {
        if (int(pq.size()) < k)
            pq.push(t);
        else if (t > pq.top()) {
            pq.pop();
            pq.push(t);
        }
    }
    vector<P> res;
    while (!pq.empty()) { res.push_back(pq.top()); pq.pop(); }
    reverse(res.begin(), res.end());
    return res;
}

int main(int argc, char* argv[]) {
    if (argc < 2) { cerr << "用法: ./lab6 k < 文本文件\n"; return 1; }
    int k = stoi(argv[1]);
    unordered_map<string, int> freq;
    countWords(freq); // 任务 2:词频字典
    vector<pair<int, string>> items;
    for (const auto& [w, c] : freq) items.push_back({c, w});

    // 任务 5:边界处理—k <= 0 或文本为空
    if (k <= 0 || items.empty()) {
        cout << "k = " << k << ", 词数 = " << items.size()
            << ":边界情形,答案为空。\\n";
        return 0;
    }

    auto r1 = topkHeap(items, k);
    auto r2 = topkStd(items, k);
    cout << "词数 = " << items.size() << ", k = " << k << "\\n";
    cout << "手写堆 : ";
    for (const auto& [c, w] : r1) cout << w << "(" << c << ") ";
    cout << "\\npriority_queue: ";
    for (const auto& [c, w] : r2) cout << w << "(" << c << ") ";
    cout << "\\n结果一致: " << (r1 == r2 ? "是" : "否") << "\\n";
    return 0;
}

```

示例输入与输出(真实运行;sample.txt 与 empty.txt 为测试文件):

```
$ cat sample.txt
to be or not to be that is the question
whether tis nobler in the mind to suffer
the slings and arrows of outrageous fortune
or to take arms against a sea of troubles
and by opposing end them to die to sleep no more
```

```
$ ./lab6 3 < sample.txt
词数 = 34, k = 3
手写堆 : to(6) the(3) or(2)
priority_queue: to(6) the(3) or(2)
结果一致: 是
```

```
$ ./lab6 5 < empty.txt
k = 5, 词数 = 0:边界情形,答案为空。
```

设计说明:① 1 基公式写进类注释,与 Python 版 0 基实现互为对照;② 元素用 (频次, 词) 元组,平局比词序,结果可复现;③ 两版结果一致即互相验证;④ 词频统计用 `unordered_map` (第5章),与堆协同完成统计 → Top-K 流水线;⑤ 边界显式处理,避免空堆访问。

下一章预告:第7章 排序——堆排序将直接使用本章的二叉堆:下滤建堆 $O(n)$,再连续 `deleteMin` n 次得到有序序列,总代价 $O(n \log n)$;插入、希尔、归并、快排逐一登场。